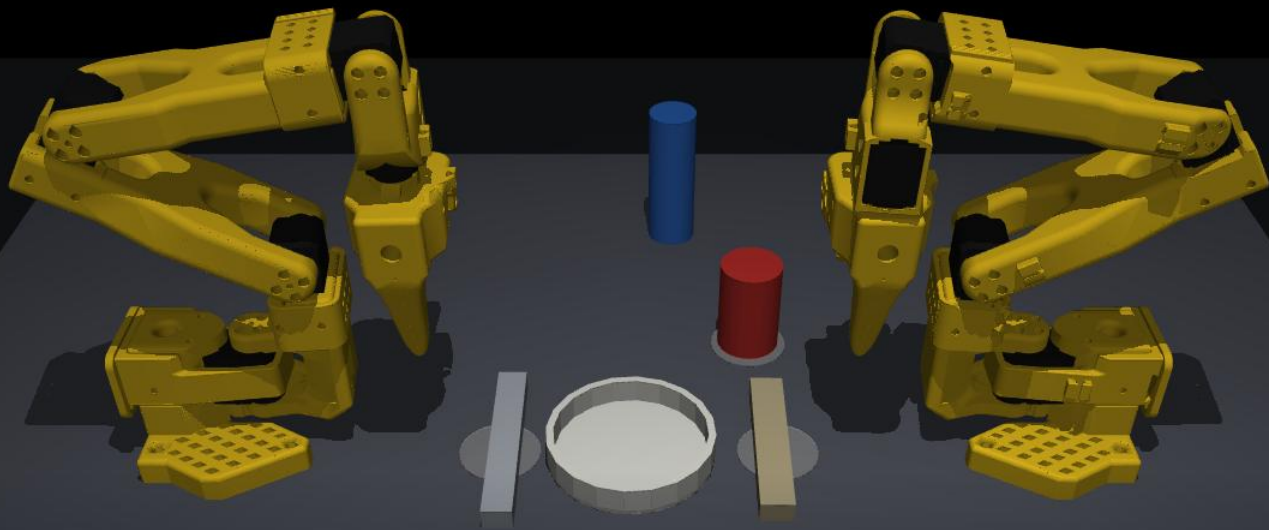




Two-Arm Table Setter

See · understand · hand over · fix mistakes — on an Intel laptop with OpenVINO

One command. Two hands. A set table.

Tested the way Intel asked: random tables, success checked against simulator truth

Say or type what you want

*"put the red thing top left of the plate and
the grey utensil on the right"*

The robot works out the rest

which item is meant, which arm, when to
pass an item across, what to do first — and
checks every step with its camera

30/30

tables set correctly

29/30

harder tables: positions,
sizes, colours

95%

trained ACT policy on
unseen tables

~4 s

to understand a command
(was 30 s)

Why it matters

Service robots for homes, hospitals, hotels and restaurants

1 Anyone can instruct it

Plain language, spoken or typed. No programming, no app, no fixed commands.

2 Two hands, real tasks

Passing, holding and placing — the everyday handling that one-armed robots can't do.

3 Checks its own work

A bump or a slip is noticed and fixed, so a job gets finished without a person watching.

4 Cheap to run and to teach

Runs on an ordinary Intel laptop with OpenVINO. Skills teach themselves from auto-recorded demos — no hand-collected data.

Observe, understand, plan, act, look again

1

Observe

overhead colour +
depth camera finds
every object

2

Understand

Qwen3-VL-4B on
OpenVINO reads the
command and the
picture

3

Plan

which arm, hand-off or
set-down swap, what
first

4

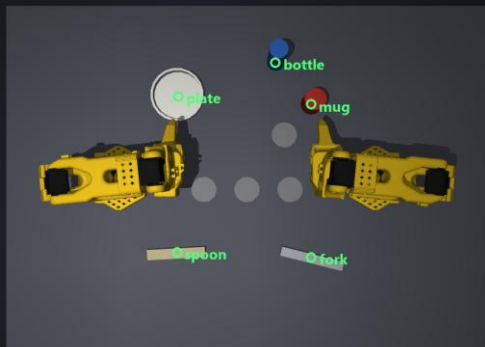
Act

two SO-101 arms; a
trained ACT policy for
the mug

5

Look again

anything knocked out of
place is fixed first

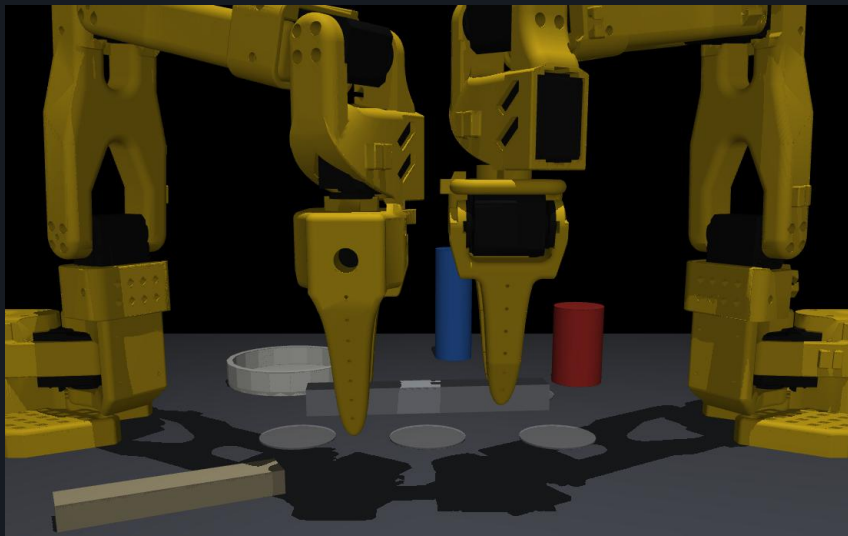


What the robot sees

Objects are found from the height map and their shape, never from the simulator. On 20 random tables every object was found, 0.5 mm average error.

Held items are tracked from the arm's own joint angles, like a real robot.

Teamwork, and it fixes its own mistakes



Hand-off

The fork starts on arm B's side but belongs on arm A's. B carries it in, A moves up at the same time, lines up 1.5 mm away, grips, then B lets go.



knocked

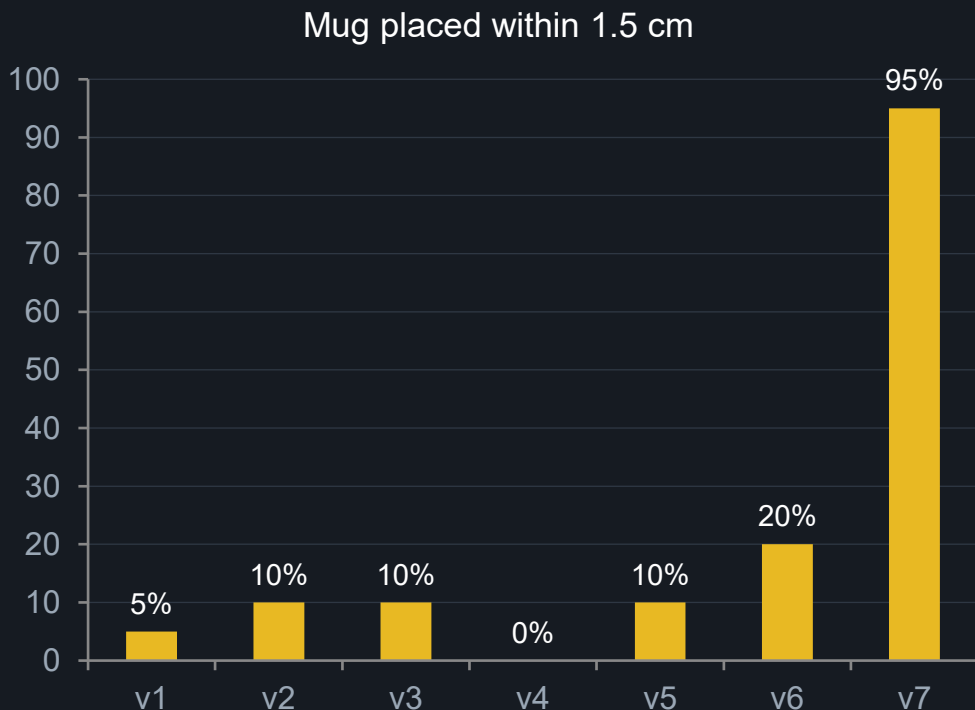
seen and fixed

Recovery

Mid-task, the plate is knocked 6 cm. After the next item the robot looks at everything it already placed, sees the plate has moved, and puts it back before carrying on.

Our own teacher trained the policy

LeRobot ACT on a laptop CPU · 1,219 demonstrations recorded automatically · success on unseen tables



1

A teacher must be consistent

picking the best of 12 grip angles looked random to the student — one fixed rule

2

Targets, not small moves

tiny per-step errors added up to centimetres

3

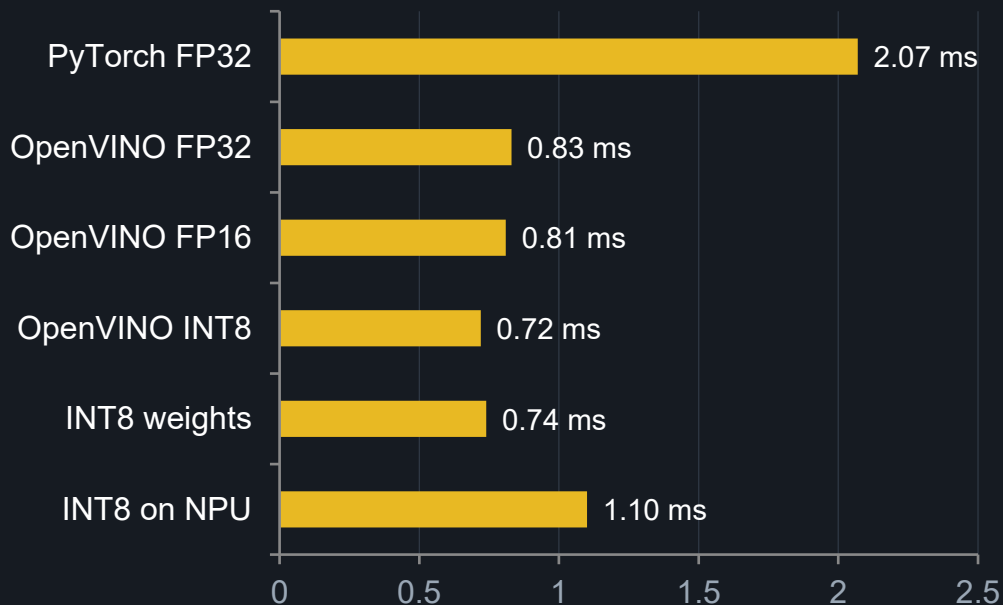
Think in the mug's frame

"grip beside the mug" becomes the same move every time

OpenVINO on all three Intel chips

Intel Core Ultra 7 155H · CPU + Arc iGPU + AI Boost NPU · OpenVINO 2026.3.1

ACT policy, ms per call, CPU unless marked (lower is better)



2.9×

faster policy on CPU with OpenVINO — task success unchanged at every precision (9–10/10)

0.90 ms

on the AI Boost NPU — 10/10 mugs placed, freeing the CPU and iGPU

1.3 s

to read a command on the Arc iGPU — 2.8× the CPU, down from 30 s originally

Open, reproducible, honest

What we'd do next

- Get the vision-language model onto the NPU — today OpenVINO's NPU compiler rejects it
- Train policies for the hand-off too; today most skills are the scripted teacher
- Add the drawer and pouring from Intel's example task

Try it

```
python brain/run.py "set the table"  
python brain/evaluate.py --seeds 10 --hard  
python bench/benchmark.py
```

github.com/PegBitStudio/two-arm-table-setter

Every scene is seeded and every number here is reproducible from one command. MIT licence. Built with MuJoCo, LeRobot, OpenVINO and Qwen3-VL.